

Seminar Report

On

**ScropIDS: A Multi-Tenant Cloud-Native Intrusion Detection
Platform with Scheduler-Driven Aggregation and
LLM-Assisted Threat Triage**

Submitted by

Konda Nagendar

Registration No. A1907506027

Bachelor of Technology

IN

Computer Science and Engineering

Under the Supervision of

[Faculty Name]

[Designation]

**LOVELY PROFESSIONAL UNIVERSITY
PUNJAB**

(April, 2026)

DECLARATION

I hereby declare that the seminar report titled *ScropIDS: A Multi-Tenant Cloud-Native Intrusion Detection Platform with Scheduler-Driven Aggregation and LLM-Assisted Threat Triage* submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a record of my own work carried out during the academic session 2025-2026.

I further declare that this report has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

I confirm that the content of this report is original and prepared by me. Any references used have been duly acknowledged. I also declare that the use of Artificial Intelligence tools, if any, has been limited to drafting support and formatting assistance, while the subject understanding, analysis, organization of ideas, and final academic responsibility remain my own.

I take full responsibility for the authenticity and originality of the work presented in this report.

Name of the Student: Konda Nagendar

Registration Number: A1907506027

Course: CSE435 - Comprehensive Seminar

Signature of the Student: _____

Date: _____

TABLE OF CONTENTS

Declaration

Chapter 1: Introduction

Chapter 2: Literature Review

Chapter 3: Conceptual Study / Seminar Work

Chapter 4: Results and Discussion

Chapter 5: Conclusion and Future Scope

Professional Profile & Repository Details

References

Chapter 1 Introduction

1.1 Title of the Seminar Topic

The topic of this seminar report is *ScropIDS: A Multi-Tenant Cloud-Native Intrusion Detection Platform with Scheduler-Driven Aggregation and LLM-Assisted Threat Triage*. The work studies how a modern intrusion detection workflow can be designed as a cloud-native platform instead of a collection of disconnected logs, scripts, and analyst-heavy review steps.

1.2 Background and Importance of the Topic

Organizations now generate large volumes of endpoint activity data through laptops, workstations, servers, and virtual machines. Process launches, authentication attempts, network connections, and system events can all provide early signs of malicious behavior. The difficulty is not only collecting these events; it is converting them into findings that are understandable, prioritized, and useful for analysts.

Traditional security monitoring stacks often separate collection, storage, analysis, and presentation into different operational tools. This fragmentation increases analyst overhead, creates inconsistent workflows, and makes it difficult to enforce clear tenant boundaries when the same monitoring platform is shared across multiple organizations. As a result, intrusion detection remains technically powerful but operationally noisy for many teams.

The importance of the seminar topic lies in its attempt to address that gap. ScropIDS combines endpoint telemetry, secure agent onboarding, tenant-aware storage, scheduled aggregation, rule-based escalation, optional large language model assistance, and dashboard visibility inside a single platform. It is therefore relevant to cybersecurity operations, cloud-native software engineering, distributed systems, and practical product design.

1.3 Objectives of the Seminar

1. Study the architecture of a modern intrusion detection platform built as a multi-tenant application.
2. Understand how endpoint telemetry can be collected, normalized, stored, and analyzed in a structured workflow.
3. Examine how scheduling, aggregation, and alert triage reduce noise compared with raw log review.
4. Review the role of optional LLM-assisted reasoning in producing explainable threat summaries.

5. Evaluate the strengths, limitations, and future scope of the ScropIDS approach.

1.4 Brief Overview of the Methodology Used

The seminar work was carried out by examining the ScropIDS repository structure, technical documentation, implementation layout, and operational workflow. Particular attention was given to the Django backend, the React dashboard, the Go-based agent starter, the scheduler-driven aggregation model, and the LLM integration layer.

The study also used a comparative approach. Existing categories such as host-based intrusion detection systems, SIEM platforms, and commercial endpoint detection solutions were reviewed at a conceptual level. Their common strengths and limitations were then compared with the design priorities visible in ScropIDS.

This report therefore combines literature-oriented discussion with repository-grounded technical analysis. It is not limited to theory, because the platform exists as a real implementation; at the same time, it is structured as a seminar report rather than an exhaustive product manual.

Chapter 2 Literature Review

2.1 Intrusion Detection as a Security Discipline

Intrusion detection systems are designed to identify suspicious activity across hosts, users, applications, or networks. Early IDS approaches were highly signature-oriented, focusing on known patterns and rule triggers. Over time, organizations adopted richer telemetry pipelines in order to capture behavior that is anomalous, repeated, or contextually risky even when no single event appears dangerous in isolation.

Two long-standing problems remain common. First, monitoring systems often generate more raw data than analysts can review effectively. Second, the operational context required to interpret alerts is frequently missing or scattered. These problems motivate the study of IDS designs that prioritize not only detection, but also explainability and workflow usability.

2.2 Host-Based IDS Platforms

Open-source host-based platforms such as OSSEC and Wazuh demonstrate the value of agent-driven monitoring. They collect endpoint-relevant data, apply rules, and surface alerts with practical security value. Their continued popularity shows that endpoint telemetry remains one of the most useful sources for early detection.

At the same time, these platforms are not always designed around lightweight multi-tenant SaaS requirements. Turning them into a tenant-aware product with self-service onboarding, per-tenant configuration, downloadable agents, and application-native dashboard flows often requires substantial customization. This limitation is one of the reasons why ScropIDS is an interesting seminar topic.

2.3 SIEM and Security Analytics Platforms

Platforms such as Splunk and Elastic Security emphasize large-scale ingestion, correlation, search, dashboarding, and alerting. They are valuable reference points because they show how central visibility and retained telemetry can improve investigations. Their operational strength is breadth, but that breadth often brings cost, tuning effort, and deployment complexity.

For smaller environments, academic projects, and focused SaaS-style security tooling, a lighter and more purpose-built system may be preferable. ScropIDS borrows the idea of centralized collection and alert review from this class of tools, while keeping the architecture narrower and easier to reason about within a student-led implementation.

2.4 Commercial EDR and AI-Assisted Triage

Commercial endpoint detection and response platforms provide mature detection logic, rich telemetry, and polished investigation workflows. They also increasingly use machine learning or guided reasoning to help analysts prioritize alerts. Their value is clear, but their internals are often closed, their licensing models are restrictive, and they are not always suitable as transparent academic study artifacts.

Recent research and industry practice also show growing interest in LLM-assisted triage, where structured telemetry is summarized into human-readable reasoning. This approach is promising because it can reduce the distance between raw event streams and analyst action. However, it must be handled carefully through strict schemas, validation, and fallback logic so that generated explanations remain operationally safe.

2.5 Research Gap and Motivation

The literature and tool landscape suggest a gap between powerful detection components and usable, tenant-aware operational delivery. Many systems can collect logs. Many systems can store data. Some systems can explain alerts. Fewer systems combine secure multi-tenant onboarding, endpoint-to-cloud ingestion, scheduler-driven aggregation, explainable alert generation, flexible support for hosted or local reasoning engines, and a unified web console.

ScropIDS addresses this gap by connecting these pieces into one platform. That is the primary reason it is a strong subject for seminar study.

Table 2.1 Comparison of common approaches and the ScropIDS focus

Approach	Common Strength	Common Limitation in This Context
Host-based IDS	Strong endpoint visibility and rule-driven app	Limited application-native multi-tenant product flow
SIEM platforms	Centralized storage, search, and dashboards	High operational complexity for focused academic c
Commercial EDR	Mature detection and response workf	Closed implementation and limited academic transpar
ScropIDS focus	Unified tenant-aware pipeline from agent to dashboard	Generalist but narrower than full enterprise EDR su

Chapter 3 Conceptual Study / Seminar Work

3.1 Overview of ScropIDS

ScropIDS is a cloud-native intrusion detection platform designed to receive normalized events from endpoint agents, process those events in tenant-scoped storage, aggregate them into meaningful windows, and generate explainable alerts for analysts. The platform is production-oriented in the sense that it is organized as a monorepo with backend, frontend, agent, and deployment components rather than a single proof-of-concept script.

The backend is built with Django and Django REST Framework. It exposes authenticated APIs for organization management, agent onboarding, event ingestion, configuration, and alert retrieval. The frontend is implemented with React, Vite, and TypeScript, providing pages for alerts, agents, rules, scheduler configuration, enrollment tokens, and LLM provider settings. The agent starter is implemented in Go so that cross-platform packaging can be supported for Windows, Linux, and macOS.

3.2 Conceptual Framework and Workflow

1. A user creates or joins an organization in the tenant-aware dashboard.
2. An enrollment token or organization access token is generated for endpoint onboarding.
3. An endpoint agent enrolls, receives credentials, and begins submitting normalized events.
4. The backend stores those events with organization ownership and validation controls.
5. The scheduler groups unprocessed events into analysis windows at configured intervals.
6. Rule logic and optional LLM analysis convert those windows into explainable findings.
7. Alerts appear in the dashboard for analyst review and action.

Conceptual Pipeline

Endpoint Agent -> Secure Ingest API -> Event Store -> Scheduler -> Aggregation -> Rules / LLM Analysis -> Alerts Dashboard

3.3 System Modules and Responsibilities

The ScropIDS seminar work revolves around several clearly separated modules whose responsibilities are visible in the repository design.

Table 3.1 Major modules studied in ScropIDS

Module	Role in the Platform
Backend API	Accepts dashboard and agent requests, applies validation, stores tenant-aware records, and aggregates
Scheduler and pipeline services	Aggregate event windows, trigger analysis, and create alerts based on threshold rules
LLM service layer	Supports OpenAI-compatible and local inference paths with strict JSON validation
Frontend dashboard	Provides visibility into alerts, agents, onboarding tokens, rules, and platform configuration
Go agent starter	Handles endpoint enrollment, configuration sync, event batching, and cross-platform packaging
Docker deployment	Supports reproducible local or pilot-scale deployment with composed services

3.4 Tools, Platforms, and Technologies Studied

The seminar topic is also significant because it brings together a complete modern stack rather than a single isolated technology. The following technologies are central to the system.

Table 3.2 Technology stack used in the platform

Technology	Use in ScropIDS
Django and DRF	Core API framework, tenancy-aware views, authentication, and serialization
PostgreSQL	Persistent structured storage for events, alerts, organizations, and configuration records
Celery and Redis	Background scheduling and asynchronous aggregation workflow
React, Vite, TypeScript	Analyst-facing dashboard and administrative interface
Go	Endpoint agent starter and cross-platform packaging workflow
Docker Compose	Reproducible deployment for integrated demonstration and testing
OpenAI-compatible APIs / Ollama	Optional model-assisted alert reasoning and triage

3.5 Multi-Tenancy, Security, and Explainability

One of the strongest conceptual features of ScropIDS is that multi-tenancy is not treated as an afterthought. Core records such as organizations, agents, events, aggregates, and alerts are tenant-scoped, which reduces the risk of operational confusion and data leakage in shared deployments. Dashboard access is resolved through user membership and organization context, while agent access uses explicit credential pairs for ingest requests.

Security controls are also present in the handling of enrollment tokens, organization access tokens, and provider secrets. The design emphasizes authenticated onboarding,

tenant ownership, and encryption for sensitive configuration such as API keys. This is important because intrusion detection platforms process operationally sensitive data and must therefore protect not only the monitored systems, but also the platform itself.

Explainability is another key seminar theme. Instead of treating model output as unrestricted free text, ScropIDS expects structured JSON responses for threat level, confidence, reasoning, and recommended action. This allows model-assisted analysis to remain bounded, reviewable, and easier to integrate with deterministic alert pipelines.

3.6 Repository Structure as Evidence of the Seminar Work

The project repository itself provides evidence that the topic is implemented beyond theory. The monorepo contains a backend application, frontend dashboard, Go agent starter, Docker deployment files, and supporting documentation such as architecture notes, API contract details, and startup guidance. This makes the seminar especially valuable because the conceptual ideas are grounded in a real engineering artifact.

Chapter 4 Results and Discussion

4.1 Key Observations Derived from the Study

The first major observation is that aggregation is central to usability. Raw telemetry is often too granular for direct analyst review, but scheduler-driven grouping turns repeated low-level events into more meaningful windows. This reduces noise and creates a better foundation for both rules and model-assisted reasoning.

The second observation is that tenant isolation influences almost every architectural decision. Once multiple organizations share the same service, ownership, query scoping, configuration boundaries, and agent onboarding must all be designed carefully. ScropIDS demonstrates this principle by making organizations a first-class part of the data model and request flow.

The third observation is that explainable summaries are operationally more valuable than alert labels alone. A threat level without reasoning still forces analysts to reconstruct the story from raw logs. ScropIDS improves this by pairing aggregation with structured reasoning fields and recommended next actions.

4.2 Conceptual Analysis of the Platform

The study suggests that ScropIDS is strongest when viewed as a workflow platform rather than only a detection engine. Its value comes from the continuity of steps: onboarding, ingest, aggregation, analysis, alerting, and dashboard review. Each of these stages is meaningful on its own, but the combined flow is what creates a practical intrusion detection experience.

Another important discussion point is flexibility. Some environments prefer hosted AI APIs for convenience, while others may require local inference for cost or privacy reasons. The dual-mode LLM support in ScropIDS is therefore not just a feature addition; it is an architectural decision that improves adaptability across different deployment contexts.

4.3 Advantages

- A clear end-to-end pipeline from endpoint telemetry to analyst-facing alerts.
- Support for Windows, Linux, and macOS agent packaging at the platform design level.
- Tenant-aware onboarding and scoped storage that suit shared deployments.
- Flexible analysis path combining deterministic rules with optional LLM assistance.
- A dashboard-driven operational experience instead of a log-only workflow.

- Reproducible deployment through containerized services.

4.4 Limitations and Constraints

The study also reveals practical limitations. ScropIDS is not yet a replacement for a mature enterprise EDR suite with kernel-level visibility, broad threat intelligence feeds, or incident response automation. The value of alerts still depends on the quality and breadth of collected endpoint events. Model-assisted triage can improve explanations, but it must remain advisory and carefully validated.

There are also natural product-maturity constraints. A starter agent and a production-oriented architecture establish a strong base, but further work is required for richer collectors, larger-scale telemetry handling, deeper analytics, and more complete role-based access controls.

4.5 Insights Gained During the Seminar

This seminar demonstrates that effective intrusion detection is as much a systems design problem as it is a security problem. Good detection requires not only rules or models, but also reliable ingestion, correct data boundaries, efficient processing intervals, usable interfaces, and human-readable outputs. ScropIDS reflects this broader understanding and is therefore a valuable case study for security-oriented software engineering.

Table 4.1 Discussion summary of ScropIDS characteristics

Aspect	Observation	Discussion
Telemetry flow	Events move from agents to a central API endpoint	Supports consistent analysis compared with host-by-host methods
Aggregation model	Scheduler creates time-window summaries	Helps reduce alert fatigue and improves contextual triage
Tenant-aware design	Organizations shape access and data boundaries	Essential for safe shared deployment and SaaS-style delivery
LLM integration	Structured reasoning augments rules	Useful when validated carefully and treated as advisory
Deployment model	Docker Compose keeps setup reproducible	Practical for academic demonstration and pilot environments

Chapter 5 Conclusion and Future Scope

5.1 Summary of the Seminar Work

This seminar studied ScropIDS as a modern intrusion detection platform that combines endpoint telemetry, cloud-native backend services, scheduled aggregation, rule-assisted escalation, LLM-assisted reasoning, and a web-based dashboard. The work demonstrates how several important software engineering and cybersecurity concepts can be integrated into a single coherent system.

5.2 Major Learning Outcomes

1. Intrusion detection becomes more practical when telemetry is structured and aggregated before review.
2. Multi-tenant architecture requires early design attention to ownership, authentication, and safe data access.
3. Explainability improves the usefulness of alerts for real human operators.
4. Modern security tools benefit from strong product thinking in addition to detection logic.
5. Cloud-native deployment and modular engineering make academic security systems easier to demonstrate and evolve.

5.3 Conclusion Drawn from the Study

The central conclusion of the study is that ScropIDS is a meaningful and technically grounded seminar topic because it moves beyond isolated log collection or purely conceptual IDS discussion. It demonstrates a complete operational path from data intake to analyst-facing output, while remaining understandable as an academic artifact. The combination of multi-tenancy, scheduler-driven aggregation, explainable alerting, and optional model-assisted reasoning gives it clear practical relevance.

5.4 Future Scope

- Deeper endpoint collectors and richer event coverage.
- MITRE ATT&CK; mapping and threat intelligence enrichment.
- More advanced detection rules and correlation strategies.
- Expanded dashboard analytics and incident response playbooks.
- Stronger RBAC and audit trails.
- Larger-scale deployment validation in more realistic environments.

Professional Profile & Repository Details

Konda Nagendar is a Computer Science and Engineering student with a strong focus on cybersecurity, secure systems, and practical offensive-defensive security workflows. The present seminar report reflects that interest by studying an intrusion detection platform that brings together cloud-native engineering, endpoint telemetry collection, alert triage, and analyst usability. The work demonstrates a strong alignment between software development and security operations.

Repository Details

GitHub Project Repository: [Add public GitHub repository URL here]

LinkedIn Profile Link: [Add LinkedIn profile URL here]

Project Title in Repository: ScropIDS

Repository Structure Covered in the Seminar: backend/, frontend/, agents/go/, docs/, and deployment files

References

1. ScropIDS project documentation: README.md, docs/architecture.md, docs/api-contract.md, and docs/start-methods.md.
2. MITRE ATT&CK; Framework. Available at: <https://attack.mitre.org/>
3. SigmaHQ Detection Rules. Available at: <https://github.com/SigmaHQ/sigma>
4. NIST Cybersecurity Framework 2.0. Available at: <https://www.nist.gov/cyberframework>
5. Django Documentation. Available at: <https://docs.djangoproject.com/>
6. Celery Documentation. Available at: <https://docs.celeryq.dev/>
7. React Documentation. Available at: <https://react.dev/>